Parte III

Administración de procesos

Capítulo 19

Concepto y estados de un proceso

En un sistema multiprogramado o de tiempo compartido, un *proceso* es la imagen en memoria de un programa, junto con la información relacionada con el estado de su ejecución.

Un programa es una *entidad pasiva*, una lista de instrucciones; un proceso es una *entidad activa*, que –empleando al programa– define la actuación que tendrá el sistema.

En contraposición con *proceso*, en un sistema por lotes se habla de *tareas*. Una tarea requiere mucha menos estructura, típicamente basta con guardar la información relacionada con la *contabilidad* de los recursos empleados. Una tarea no es interrumpida en el transcurso de su ejecución. Ahora bien, esta distinción no es completamente objetiva —y se pueden encontrar muchos textos que emplean indistintamente una u otra nomenclatura.

Si bien el sistema brinda la *ilusión* de que muchos procesos se están ejecutando al mismo tiempo, la mayor parte de ellos típicamente está esperando para continuar su ejecución —en un momento determinado sólo puede estar ejecutando sus instrucciones un número de procesos igual o menor al número de procesadores que tenga el sistema.

En este capítulo se desarrollan los conceptos relacionados con procesos, hilos, concurrencia y sincronización —las técnicas y algoritmos que emplea el sistema operativo para determinar cómo y en qué orden hacer los cambios de proceso que brindan al usuario la ilusión de simultaneidad se abordarán en el capítulo IV.

19.1. Estados de un proceso

Un proceso, a lo largo de su vida, alterna entre diferentes *estados* de ejecución. Éstos son:

Nuevo Se solicitó al sistema operativo la creación de un proceso, y sus recursos y estructuras están siendo creadas.

Listo Está listo para iniciar o continuar su ejecución pero el sistema no le ha asignado un procesador.

En ejecución El proceso está siendo ejecutado en este momento. Sus instrucciones están siendo procesadas en algún procesador.

Bloqueado En espera de algún evento para poder continuar su ejecución (aun si hubiera un procesador disponible, no podría avanzar).

Zombie El proceso ha finalizado su ejecución, pero el sistema operativo debe realizar ciertas operaciones de limpieza para poder eliminarlo de la lista.[1]

Terminado El proceso terminó de ejecutarse; sus estructuras están a la espera de ser *limpiadas* por el sistema operativo.

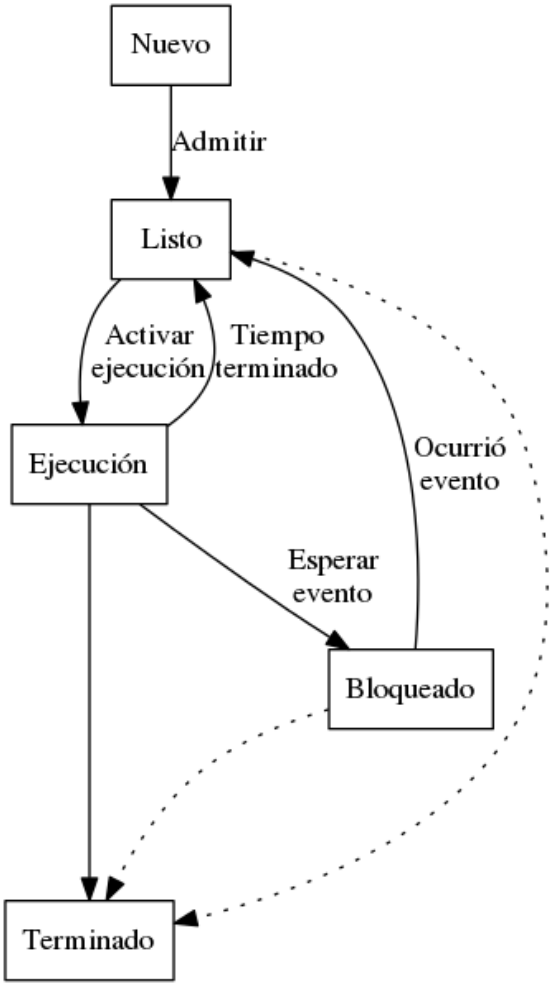


Figura 19.1: Diagrama de transición entre los estados de un proceso.

19.2. Información asociada a un proceso

La información que debe manipular el sistema operativo relativa a cada uno de los procesos actuales se suele almacenar en una estructura llamada *bloque de control de proceso* (PCB - *Process Control Block*). El PCB incluye campos como:

[1] Estas operaciones pueden incluir notificar al proceso padre, cerrar las conexiones de red que tenía activas, liberar memoria, etcétera.

Estado del proceso El estado actual del proceso.

Contador de programa Cuál es la siguiente instrucción a ser ejecutada por el proceso.

Registros del CPU La información específica del estado del CPU mientras el proceso está en ejecución (debe ser respaldada y restaurada cuando se registra un cambio de estado).

Información de planificación (scheduling) La prioridad del proceso, la *cola* en que está agendado, y demás información que puede ayudar al sistema operativo a planificar los procesos; se profundizará en este tema en el capítulo IV.

Información de administración de memoria La información de mapeo de memoria (páginas o segmentos, dependiendo del sistema operativo), incluyendo la pila (*stack*) de llamadas. Se abordará el tema en el capítulo V.

Información de contabilidad Información de la utilización de recursos que ha tenido este proceso —puede incluir el tiempo total empleado y otros (*de usuario*, cuando el procesador va avanzando sobre las instrucciones del programa propiamente, *de sistema* cuando el sistema operativo está atendiendo las solicitudes del proceso), uso acumulado de memoria y dispositivos, etcétera.

Estado de E/S Listado de dispositivos y archivos asignados que el proceso tiene *abiertos* en un momento dado.

Capítulo 20

Procesos e hilos

Como se vio, la cantidad de información que el sistema operativo debe manejar acerca de cada proceso es bastante significativa. Si cada vez que el *planificador* elige qué proceso pasar de *Listo* a *En ejecución* debe considerar buena parte de dicha información, la simple transferencia de todo esto entre la memoria y el procesador podría llevar a un desperdicio *burocrático*[1] de recursos. Una respuesta a esta problemática fue la de utilizar los *hilos de ejecución*, a veces conocidos como *procesos ligeros* (LWP, *Lightweight processes*).

Cuando se consideran procesos basados en un modelo de hilos, se puede proyectar en sentido inverso que todo proceso es como un solo hilo de ejecución. Un sistema operativo que no ofreciera soporte expreso a los hilos los planificaría exactamente del mismo modo.

Pero visto desde la perspectiva del proceso hay una gran diferencia: si bien el sistema operativo se encarga de que cada proceso tenga una visión de virtual exclusividad sobre la computadora, todos los hilos de un proceso comparten un sólo espacio de direccionamiento en memoria y los archivos y dispositivos abiertos. Cada uno de los hilos se ejecuta de forma (aparentemente) secuencial y maneja su propio contador de programa y pila (y algunas estructuras adicionales, aunque mucho más ligeras que el PCB).

[1] Entendiendo *burocrático* como el tiempo que se pierde en asuntos administrativos. Recordar que el tiempo que consume el sistema operativo en administración es tiempo perdido para el uso real, productivo del equipo.

20.1. Los hilos y el sistema operativo

La programación basada en hilos puede hacerse completamente y de forma transparente en espacio de usuario (sin involucrar al sistema operativo). Estos hilos se llaman *hilos de usuario* (*user threads*), y muchos lenguajes de programación los denominan *hilos verdes* (*green threads*). Un caso de uso interesante es en los sistemas operativos mínimos (p. ej. para dispositivos embebidos), capaces de ejecutar una máquina virtual (ver sección F.1) de alguno de esos lenguajes: si bien el sistema operativo no maneja multiprocesamiento, mediante los hilos de usuario se crean procesos con multitarea interna.

Los procesos que implementan hilos ganan un poco en el rendimiento gracias a no tener que reemplazar al PCB activo cuando intercalan la ejecución de sus diferentes hilos; pero además de esto, ganan mucho más por la ventaja de compartir espacio de memoria sin tener que establecerlo explícitamente a través de *mecanismos de comunicación entre procesos* (IPC, *Inter Process Communications*). Dependiendo de la plataforma, a veces los hilos de usuario inclusive utilizan multitarea cooperativa para pasar el control dentro de un mismo proceso. Cualquier llamada al sistema *bloqueante* (como obtener datos de un archivo para utilizarlos inmediatamente) interrumpirá la ejecución de *todos* los hilos de ese proceso, dado que el control de ejecución es entregado al sistema operativo quien en este caso no conoce nada sobre los hilos.

Continuando con el desarrollo histórico de este mecanismo, el siguiente paso fue la creación de hilos *informando* al sistema operativo, típicamente denominados *hilos de kernel* (*kernel threads*). Esto se hace a través de bibliotecas de sistema que los implementan de forma estándar para los diferentes sistemas operativos o arquitecturas (p. ej. `pthreads` para POSIX o `Win32_Thread` para Windows). Estas bibliotecas aprovechan la comunicación con el sistema operativo tanto para solicitudes de recursos (p. ej. un proceso basado en hilos puede beneficiarse de una ejecución verdaderamente paralela en sistemas multiprocesador) como para una gestión de recursos más comparable con una situación de multiproceso estándar.

20.2. Patrones de trabajo con hilos

Hay tres patrones en los que caen generalmente los modelos de hilos; se puede emplear más de uno de estos patrones en diferentes áreas de cada aplicación, e incluso se pueden *anidar* (esto es, se podría tener una *línea de ensamblado* dentro de la cual uno de los pasos sea un *equipo de trabajo*):

Jefe/trabajador Un hilo tiene una tarea distinta de todos los demás: el hilo *jefe* genera o recopila tareas para realizar, las separa y se las entrega a los hilos *trabajadores*.

Este modelo es el más común para procesos que implementan servidores (es el modelo clásico del servidor Web *Apache*) y para aplicaciones gráficas (GUI), en que hay una porción del programa (el hilo *jefe*) esperando a que ocurran eventos externos. El jefe realiza poco trabajo, se limita a *invocar* a los trabajadores para que hagan el trabajo *de verdad*; como mucho, puede llevar la contabilidad de los trabajos realizados.

Típicamente, los hilos trabajadores realizan su operación, posiblemente notifican al *jefe* de su trabajo, y *finalizan* su ejecución.

Equipo de trabajo al iniciar la porción multihilos del proceso, se crean muchos hilos idénticos, que realizarán las mismas tareas sobre diferentes datos. Este modelo es frecuentemente utilizado para cálculos matemáticos (p. ej.: criptografía, render, álgebra lineal). Puede combinarse con un estilo jefe/trabajador para irle dando al usuario una previsualización del resultado de su cálculo, dado que éste se irá ensamblando progresivamente, pedazo por pedazo.

Su principal diferencia con el patrón *jefe/trabajador* consiste en que el trabajo a realizar por cada uno de los hilos se plantea desde principio, esto es, el paso de *división de trabajo* no es un hilo más, sino que prepara

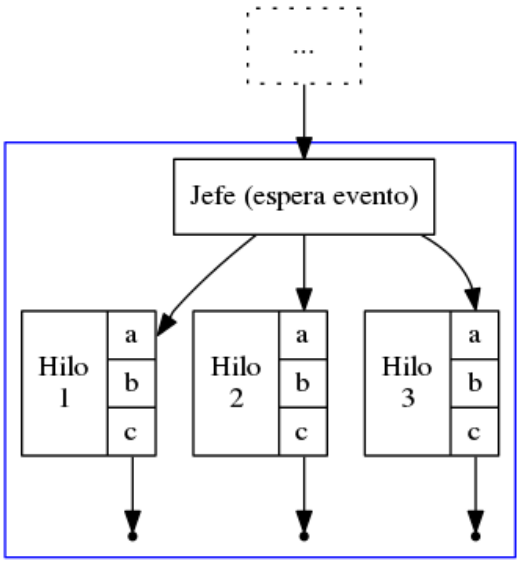


Figura 20.1: Patrón de hilos jefe/trabajador.

los datos para que éstos sean lanzados en *paralelo*. Estos datos no son resultado de *eventos independientes* (como en el caso anterior), sino partes de un solo cálculo. Como consecuencia, resulta natural que en este modelo los resultados generados por los diferentes hilos son *agregados* o *totalizados* al terminar su procesamiento. Los hilos no *terminan*, sino que *son sincronizados* y luego continúan la ejecución lineal.

Línea de ensamblado si una tarea larga puede dividirse en pasos sobre bloques de la información total a procesar, cada hilo puede enfocarse a hacer sólo un paso y pasarle los datos a otro hilo conforme vaya terminando. Una de las principales ventajas de este modelo es que ayuda a mantener rutinas simples de comprender, y permite que el procesamiento de datos continúe, incluso si parte del programa está bloqueado esperando E/S.

Un punto importante a tener en cuenta en una línea de ensamblado es que, si bien los hilos trabajan de forma secuencial, pueden estar ejecutándose paralelamente sobre bloques consecutivos de información, eventos, etcétera.

Este patrón es claramente distinto de los dos anteriormente presentados; si bien en los anteriores los diferentes hilos (a excepción del hilo *jefe*) eran casi siempre idénticos (aplicando las mismas operaciones a distintos conjuntos de datos), en este caso son todos completamente distintos.

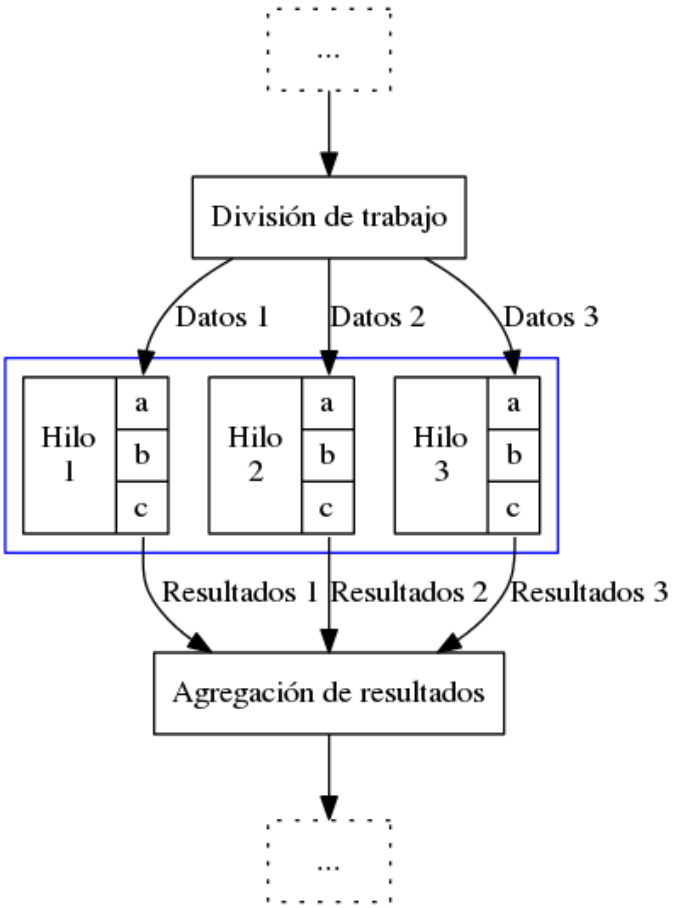


Figura 20.2: Patrón de hilos *Equipo de trabajo*.

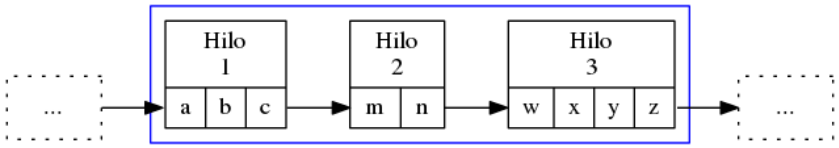


Figura 20.3: Patrón de hilos *Línea de ensamblado*.

Capítulo 21

Concurrencia

21.1. Introducción

Desde un punto de vista formal, la *concurrencia* no se refiere a dos o más eventos que ocurren a la vez sino a dos o más eventos cuyo orden es *no determinista*, esto es, eventos acerca de los cuales *no se puede predecir el orden relativo en que ocurrirán*. Si bien dos procesos (o también dos hilos) completamente independientes

entre sí ejecutándose simultáneamente son concurrentes, los temas que en la presente sección se expondrán se ocupan principalmente de procesos cuya ejecución está vinculada de alguna manera (p. ej.: dos procesos que comparten cierta información o que dependen uno del otro).

Aunque una de las tareas principales de los sistemas operativos es dar a cada proceso la ilusión de que se está ejecutando en una computadora dedicada, de modo que el programador no tenga que pensar en la competencia por recursos, a veces un programa requiere interactuar con otros: parte del procesamiento puede depender de datos obtenidos en fuentes externas, y la cooperación con hilos o procesos externos es fundamental.

Se verá que pueden aparecer muchos problemas cuando se estudia la interacción entre hilos del mismo proceso, la sincronización entre distintos procesos, la asignación de recursos por parte del sistema operativo a procesos simultáneos, o incluso cuando interactúan usuarios de diferentes computadoras de una red —se presentarán distintos conceptos relacionados con la concurrencia utilizando uno de esos escenarios, pero muchos de dichos conceptos en realidad son independientes del escenario: más bien esta sección se centra en la relación entre procesos que deben compartir recursos o sincronizar sus tareas.

Para presentar la problemática y los conceptos relacionados con la concurrencia suelen utilizarse algunos problemas clásicos, que presentan casos particulares muy simplificados, y puede encontrárseles relación con distintas cuestiones que un programador enfrentará en la vida real. Cada ejemplo presenta uno o más conceptos. Se recomienda comprender bien el ejemplo, el problema y la solución y desmenuzar buscando los casos límite como ejercicio antes de pasar al siguiente caso. También podría ser útil imaginar en qué circunstancia un sistema operativo se encontraría en una situación similar.

Para cada problema se mostrará *una forma* de resolverlo aunque en general hay más de una solución válida. Algunos de éstos fueron originalmente planteados como justificación del desarrollo de las estructuras de control presentadas o de nuevos paradigmas de concurrencia y muchos son aún objeto de debate e investigación.

Para profundizar más en este tema, se recomienda el libro The little book of semaphores (Downey 2008). En este libro (de libre descarga) se encuentran muchos ejemplos que ilustran el uso de semáforos, no sólo para resolver problemas de sincronización, sino como un mecanismo simple de comunicación entre procesos. También se desarrollan distintas soluciones para los problemas clásicos (y no tan clásicos).

21.2. Problema: el jardín ornamental

21.2.1. Descripción del problema

Un gran jardín ornamental se abre al público para que todos puedan apreciar sus fantásticas rosas, arbustos y plantas acuáticas. Por supuesto, se cobra una módica suma de dinero a la entrada para lo cual se colocan dos torniquetes, uno en cada una de sus dos entradas. Se desea conocer cuánta gente ha ingresado al jardín así que se instala una computadora conectada a ambos torniquetes: estos envían una señal cuando una persona ingresa al jardín. Se realiza un modelo simplificado de la situación, así que no se estudiarán los detalles del hardware utilizado. Aquí es importante notar que los dos torniquetes son objetos que existen y se comportan en paralelo e independientemente: los eventos que generan no tienen un orden predecible. Es decir, que cuando se escriba el software no se sabe en qué momento llegará cada visitante ni qué torniquete utilizará.

Se simulará un experimento en el que 20 visitantes ingresan por cada torniquete. Al final de la simulación deberá haber 40 visitantes contados. Una implementación tentativa podría ser la siguiente:[1]

```
int cuenta;
```

[1] Se utiliza una versión ficticia del lenguaje C para el ejemplo, evitando entrar en los detalles de sintáxis de un lenguaje concurrente.